Boosting

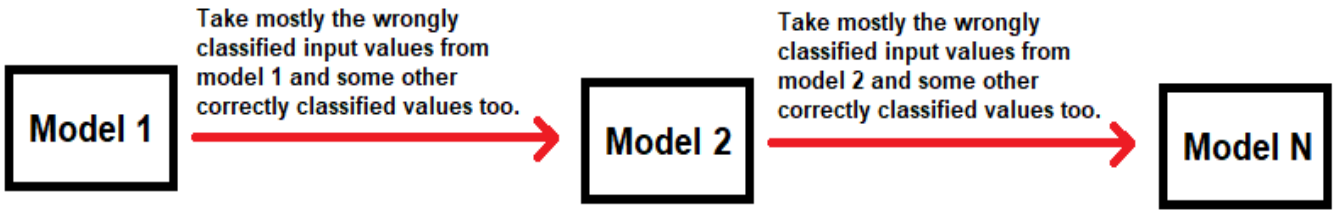
Boosting is combining weak models to create one single strong model. There are 3 types of boosting : Adaboost, Gradient Boosting, XGBoost and Catboost. We'll talk about Adaboost here first.

Adaboost Classification Introduction

In bagging we use multiple models simultaneously and independently. No model depends on another model for anything. If one model makes any error in prediction, the other model doesn't know about it.
But in adaboost, at first we train one model with the main dataset and whatever that model makes wrong predictions on those input *values*|*rows*, we tell the next|second model

> "Hey! Your previous model, the first model, made wrong predictions on those input values. So you train yourself specifically on those wrongly classified input values and some other correctly classified input values as well to see how you correctly classify them even though your previous model already correctly classified them but I want to see your performance on some of the correctly classified input values too".

Then again if the second model makes wrong predictions on some input values, then those wrongly classified input values and some other correctly classified input values are given to the third model to train. This keeps going on for all the models in adaboost. Then at the end to predict the classification for an input value we use an equation(not majority voting).



As you can see, models are **sequentially** added after one another.

By default we use Decision Tree in adaboost (mostly we'll use Decision Tree in adaboost because this gives better results 99% times) and each decision tree model has depth only 1 i.e. max_depth = 1. A decision tree with max_depth = 1 has High Bias since the inability of the model to understand the data points is high with depth = 1 AND since bias is high, so Variance is low. A decision tree with depth = 1 is called **Decision Stump**. So in adaboost we'll address Decision Stump instead of Decision Tree.

The way we mostly give wrongly classified input rows|values alongside some correctly classified input values to the next model, is called **upsampling**.

Adaboost is called "**Stagewise Additive Method (SAMME)**" because adaboost adds(additive method) all the weak learners and it adds weak learners one by one(stagewise).

How do we predict for binary classification?

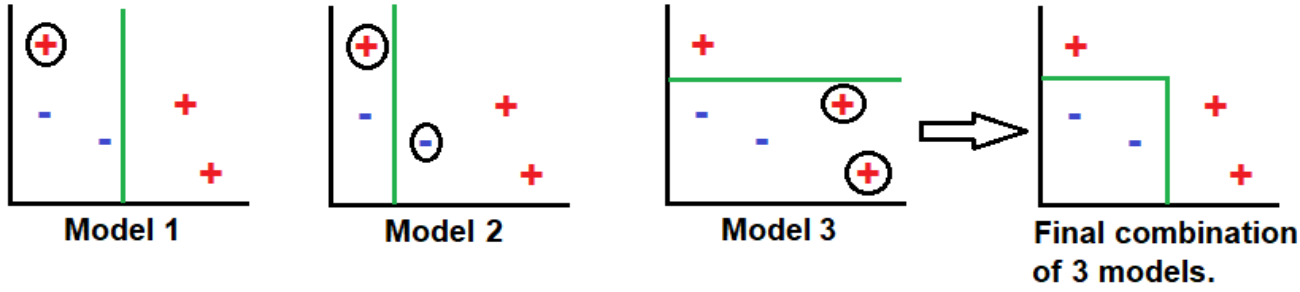
Each model returns an alpha value which denotes the Total Error done by this model. Also for binary classification the output is +1(positive) or -1(negative). Now assume for an input value|row, we want to perform prediction and we've 2 models in adaboost :

alpha1 * Model_1's prediction_value + alpha2 * Model_2's prediction_value +
= 0.20 * 1 + 0.30 * (-1) [values are assumed]
= - 0.1, since its negative value, the output is Negative.

Visualization on prediction for binary classification :

Since each model with depth = 1 can have BEST split|cut only once on any one column and then the model is done, (below each picture is taken from campusx)

Now watch from [6:11 till end]. For simplicity purposes in this video the 3 models worked on the same dataset. The 1st model made its ONE TIME BEST SPLIT, similarly the other two models did and then finally we combined all the splits into one. E.g. another one below :



How to calculate for adaboost step by step :

Watch [AdaBoost - A Step by Step Explanation]. But still I am going to write explanation from my own from this tutorial :

Steps for each model

At first we initialize the weight for each input value|row by 1 / row_numbers. Then the main steps start :

1) Prediction (ŷ) for the current model and calculate alpha (α) for that model.
2) Update the weight for misclassified and classified points.
3) Upsampling, creating and sending the new dataset to the next model which the next model will be trained on.

Initialize the weight for each input value|row: (assume we've 3 decision tree models in adaboost)

x1	x2	y	weight
3	7	1	0.2
2	9	0	0.2
1	4	1	0.2
9	8	0	0.2
3	7	0	0.2

1) Prediction (ŷ) for the current first model and calculate alpha (α) for that model :

x1	x2	y	weight	ŷ
3	7	1	0.2	1
2	9	0	0.2	1
1	4	1	0.2	0
9	8	0	0.2	0
3	7	0	0.2	0

$$\alpha = 0.5 * \ln\left(\frac{1 - \text{Total_error}}{\text{Total_error}}\right)$$

Ŷ made 2 wrong predictions. So total_error = Total **weight** of all the errors
= 0.2 + 0.2 = 0.4

So, α_1 = 0.5 * ln((1 - 0.4) / 0.4) = 0.2.

(α = 0.5 * ln((1 - 0.1) / 0.1) = 1.09.)

The fewer errors a model makes during training, the more the alpha value is, the more that model can give good predictions. So, in adaboost Alpha serves as a measure of the contribution of each *weak learner (decision stump*) to the AdaBoost's final ensemble prediction.
Alpha is the weight of each weak model in AdaBoost. The more the weight, the better that weak model performed to its corresponding dataset where wrongly classified values are more.

2) Update the weight for misclassified and classified points :

x1	x2	y	weight
3	7	1	0.16
2	9	0	0.24
1	4	1	0.24
9	8	0	0.16
3	7	0	0.16

For misclassified points :

$$\text{new_weight} = \text{old_weight} * e^{\alpha}$$

For classified points :

$$\text{new_weight} = \text{old_weight} * e^{-\alpha}$$

Sum(weight) = 0.16 + 0.16 + 0.16 + 0.24 + 0.24 = 0.96.

To make sure the sum is 1, we normalize it by dividing each new weight with the sum of weight :

x1	x2	y	weight
3	7	1	0.167
2	9	0	0.25
1	4	1	0.25
9	8	0	0.167
3	7	0	0.167

normalized_weight
= weight / sum(weight)

0.167 + 0.167 + 0.167 + 0.25 + 0.25 = 1

3) Upsampling, creating and sending the new dataset to the next model which the next model will be trained on :

At first we need to create cumulative_range :

index	x1	x2	y	weight	cumulative_range
0	3	7	1	0.167	0 - 0.167
1	2	9	0	0.25	0.167 - 0.417
2	1	4	1	0.25	0.417 - 0.667
3	9	8	0	0.167	0.667 - 0.834
4	3	7	0	0.167	0.834 - 1

Since we have 5 rows, we'll now generate 5 numbers from 0 to 1 randomly. Assume they are = [0.71, 0.66, 0.48, 0.44, 0.94] (generated by numpy) and they belong to each cumulative_range and their indexes are = [3, 2, 2, 2, 4].

As you can see that index 1 and 2 has a **bigger range** and that's why when we randomly generate numbers here, there's a high chance that most random values will fall under those misclassified cumulative_range because their new **weights** are larger than others. So the new dataset which we just created by **upsampling**(by randomly generating values from 0 to 1 and picking their corresponding index) to give it to the next second model :

index	x1	x2	y	weight
3	9	8	0	0.167
2	1	4	1	0.25
2	1	4	1	0.25
2	1	4	1	0.25
4	3	7	0	0.167

Now continue these 3 steps for the other models sequentially. At the end we'll get α_1, α_2, α_3 for the 3 decision stumps we used in adaboost which we need for predicting for each value|row.

Bagging vs Boosting

Watch Difference between Bagging and Boosting.

After watching that tutorial you may think how the weight of each model in bagging is the same.

By assigning weight in Adaboost, we can know which (weak) model has greater importance in final prediction according to its definition.

But in bagging everyone has the same importance. Doesn't matter if you've performed best during training or not. It's all up to majority voting. Whatever the majority says the answer is that.